



Q-Learning vs. SARSA for ICU Sepsis Treatment

A Tabular RL Study on the ICU-Sepsis Benchmark

Group 11: Shuai Yuan · Scarlett Shi · Friedemann Schmidt · Ananya Jhamb

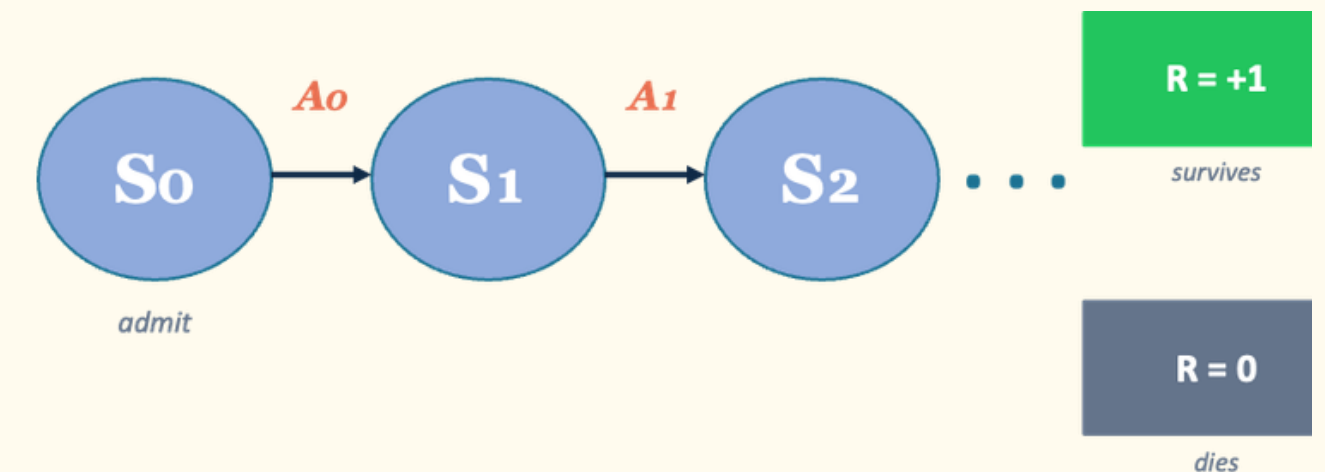
Sepsis: A Sequential Decision Problem



What is Sepsis?

- A life-threatening organ dysfunction caused by the body's dysregulated response to infection
- Requiring **continuous**, personalized treatment as the patient's condition **evolves**.
- **1 in 5 deaths** globally involve Sepsis

One Episode in the ICU



Clinicians choose actions (IV fluids + vasopressors); patient state evolves; outcome arrives only at the end.

Treatment is a sequence — not a single prediction.

Why Reinforcement Learning?

Three properties of sepsis treatment make supervised learning unsuitable — and RL the natural choice.

01. Evaluative, not instructive

We learn whether the patient survived — never what the optimal dose would have been. No supervised labels exist.

02. Delayed feedback

Reward signal arrives only at the terminal state: +1 for survival, 0 for death. All intermediate rewards are zero.

03. Sequential & state-evolving

Each action affects future patient states. The decision process maps directly onto a Markov decision process.

Our project

Q-Learning (off-policy) vs. SARSA (on-policy)

How do these foundational tabular methods behave in a standardized clinical MDP?

The ICU-Sepsis MDP

Built from records of ~17,000 sepsis patients — already preprocessed & standardized

716 states

K-means clusters of 47 patient features (vitals, labs, demographics) measured in 4-hour blocks.

25 actions

A 5×5 grid of dosage levels: intravenous fluids $\times$ vasopressors.

0/+1 rewards

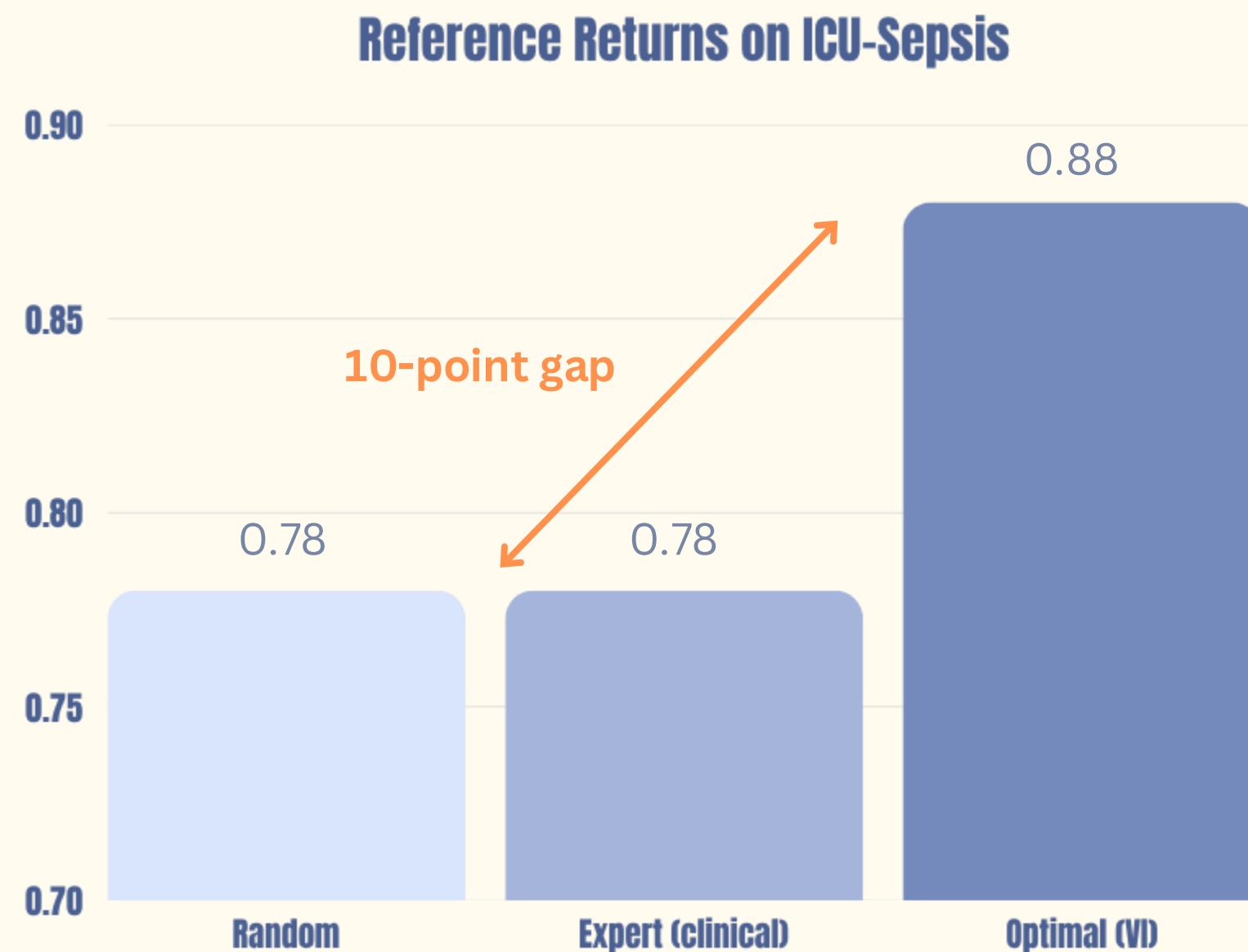
Sparse, terminal: 0 throughout, +1 only on patient survival, 0 on death.

$\gamma=1$ discount

Undiscounted — expected return equals the probability of patient survival.

Maximizing expected return = maximizing patient survival probability.

Baselines & Learning Gap



The Headroom

0.78 → 0.88

The window our Q-Learning and SARSA agents need to climb into

Why this benchmark over raw MIMIC-III?

- Standardized — results comparable across groups
- No data-use agreement, no preprocessing burden
- Stays tabular
- Lets us focus on RL methodology

Implementation Pipeline

Environment

- Official ICU-Sepsis benchmark environment
- 716 patient states
- 25 treatment actions: IV fluids × vasopressors
- Terminal reward: +1 survival, 0 death

Agent

- Q-table: 716×25
- ϵ -greedy exploration
- ϵ decays from 1.0 to 0.001
- $\gamma = 1.0$, so return represents survival probability

TD Update Target

Q-Learning, off-policy:

$$r + \gamma \cdot \max_{a'} Q(s', a')$$

Bootstraps from the best possible next action.

SARSA, on-policy:

$$r + \gamma \cdot Q(s', a')$$

Bootstraps from the action actually selected.

All hyperparameters were passed as function arguments rather than hardcoded, so the same codebase could be reused for tuning and multi-seed runs.

Correctness Checks

Before hyperparameter tuning, we ran sanity checks to confirm the implementation is correct.

Random Policy Test

Average return = 0.7810

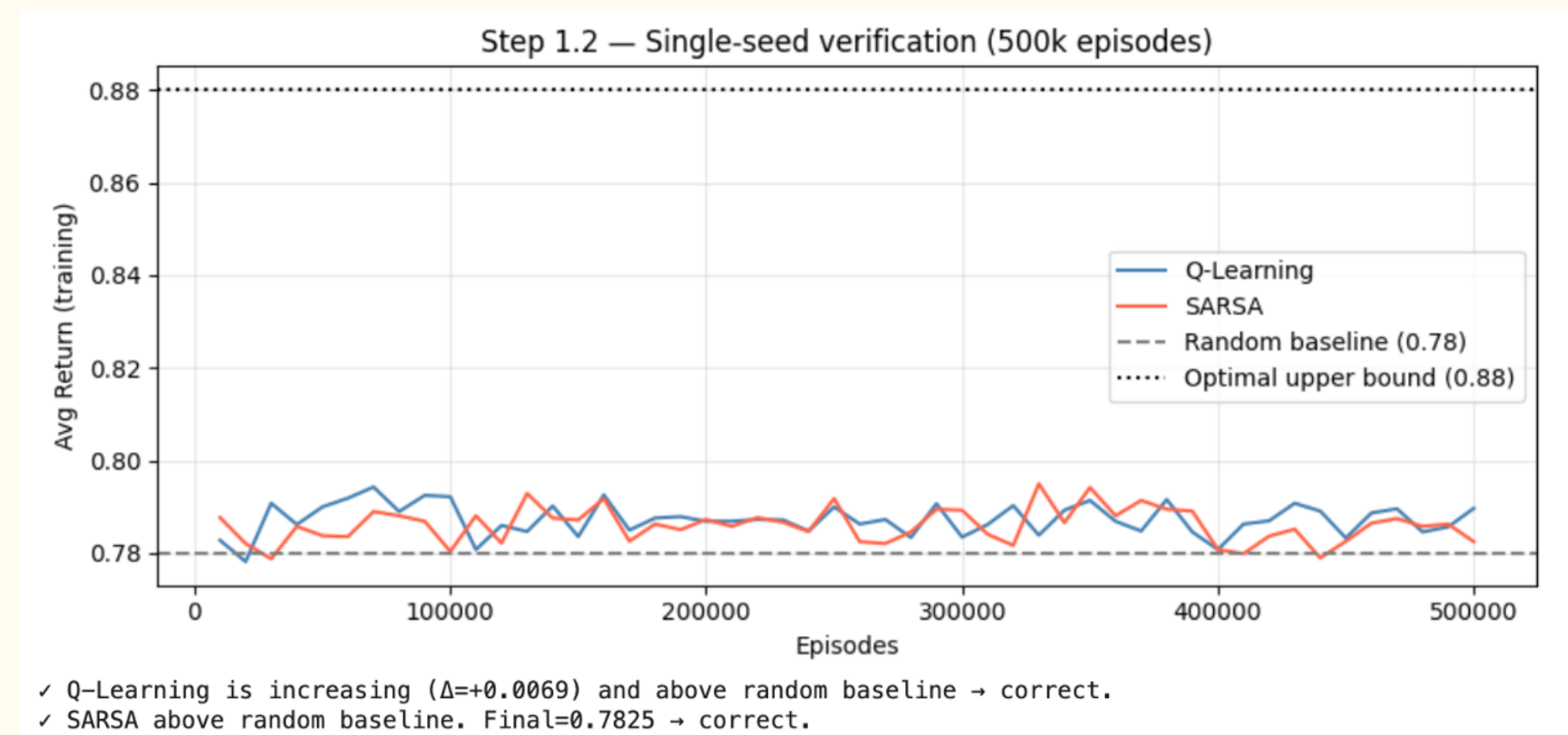
Paper baseline ≈ 0.78

Single-Seed Training

500k episodes, seed = 5

Q-Learning = 0.7897

SARSA = 0.7825



The random baseline match validates the environment and reward scale; the single-seed runs confirm that both Q-value update loops execute correctly before tuning.

Hyperparameter Tuning

Simplified grid search used to choose stable settings

Tuning setup

- Learning rates: 0.001, 0.0025, 0.01
- Exploration fractions: 0.10 and 0.25
- 3 random seeds: 5, 15, 25
- 500k episodes per configuration

Best Q-Learning config

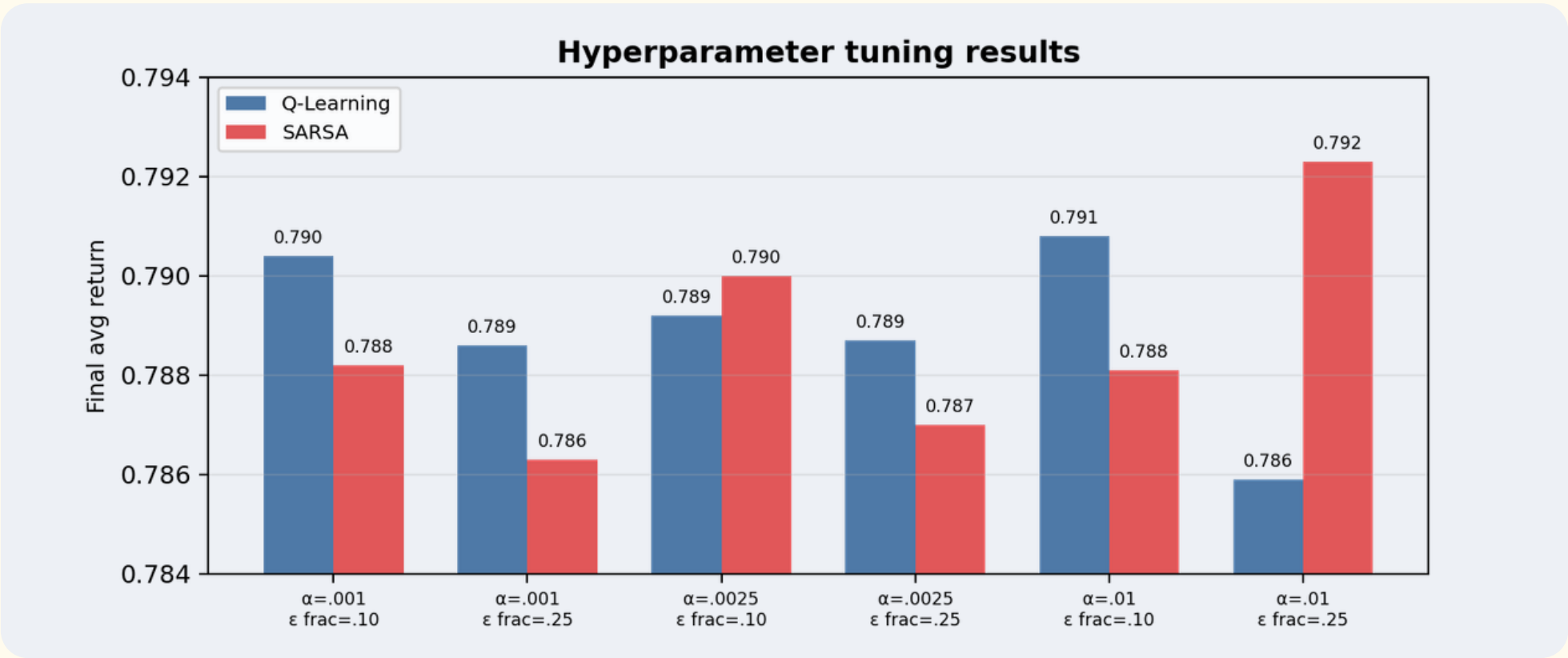
$\alpha = 0.01, \epsilon \text{ frac} = 0.10$

Final avg return:
0.7908

Best SARSA config

$\alpha = 0.01, \epsilon \text{ frac} = 0.25$

Final avg return:
0.7923



Comparison with the paper

Method	Paper setting / return	Our setting / return
Q-Learning	$\alpha=0.0025, \epsilon \text{ frac}=0.10$ reported ≈ 0.84	$\alpha=0.01, \epsilon \text{ frac}=0.10$ 0.7908
SARSA	$\alpha=0.0025, \epsilon \text{ frac}=0.25$ reported ≈ 0.79	$\alpha=0.01, \epsilon \text{ frac}=0.25$ 0.7923

Beyond tabular RL: DQN Implementation

Replace the Q-table with a neural-network Q-function.

Network architecture



01. DQN components

- Online Q-network + target network
- Stable TD targets through delayed updates
- Replay buffer = 10,000
- Target update every 512 steps
- Polyak soft update: $\tau = 0.01$

02. Training setup

- Adam optimizer, learning rate = 0.001
- Batch size = 64
- Learning starts after 10,000 steps
- Train every 10 environment steps

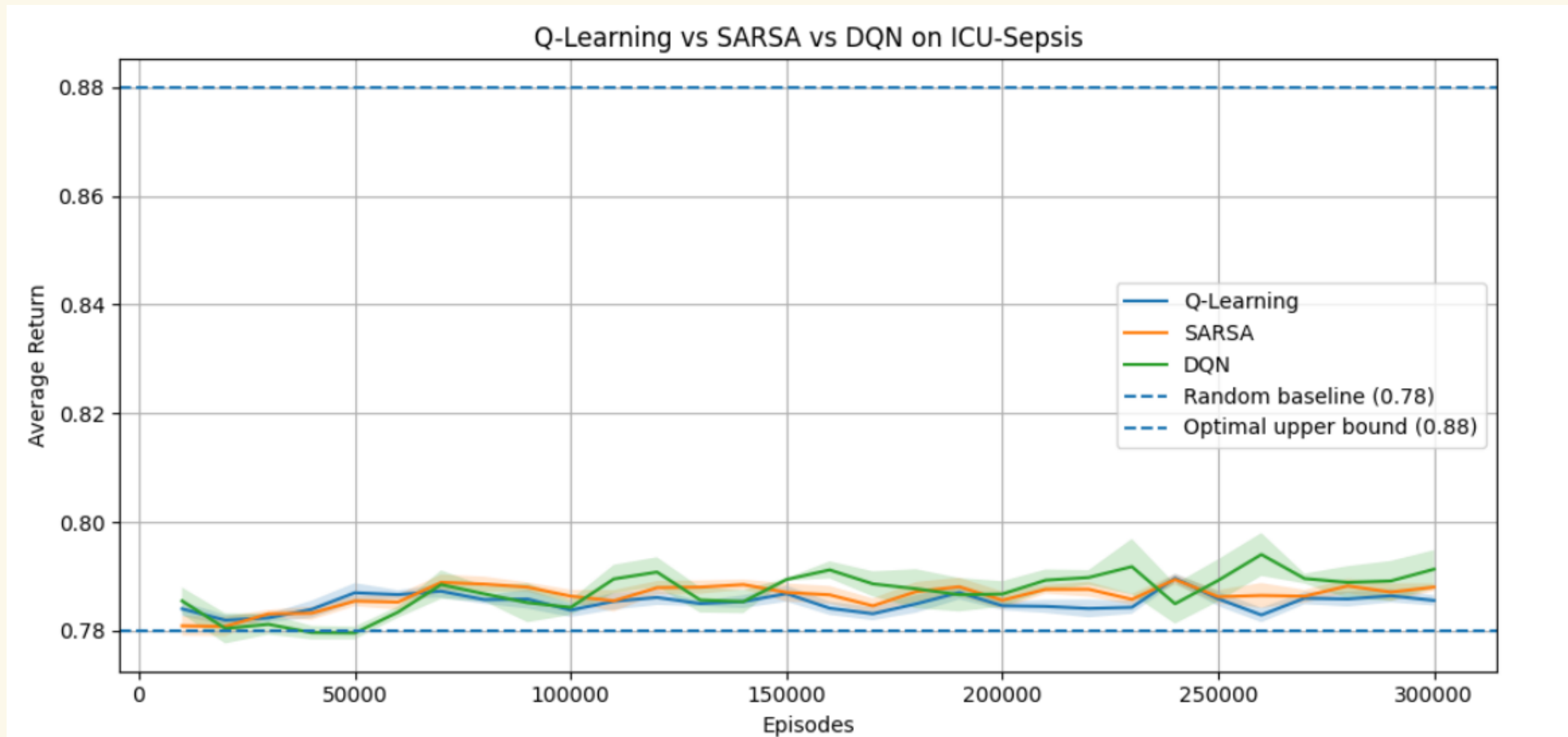
03. Run protocol

- ϵ -greedy: 1.0 $\rightarrow$ 0.001
- Exploration fraction = 0.25
- 3 seeds: 0, 1, 2
- 500k episodes per seed

Trade-off: More scalable than tabular Q-learning but adds complexity with limited gain in a 716-state MDP

Multi-Seed Formal Experiments - Result

Q-Learning leads early, SARSA stabilizes, and DQN achieves the best long-term performance



Multi-Seed Formal Experiments - Result

Q-Learning leads early, SARSA stabilizes, and DQN achieves the best long-term performance

Method	Episodes	Eval Interval	# Seeds	Final Performance (Mean ± SE)	Early Stage (≤10k)	Late Stage Performance
Q-Learning	300,000	10,000	10	0.7856 ± 0.0009	Best	Overtaken by SARSA & DQN
SARSA	300,000	10,000	10	0.7880 ± 0.0007	Moderate	Stable, slightly better than QL
DQN	300,000	10,000	3	0.7913 ± 0.0035	Worst	Best overall performance

Learning Behavior & Conclusion

Q-Learning starts fast, SARSA stabilizes, and DQN achieves the best long-term performance

01. Learning behavior

- **Early stage:** Q-Learning performs best → rapid reward propagation
- **Mid stage:** SARSA overtakes Q-Learning → improved stability
- **Late stage:** DQN achieves highest returns → benefits from generalization

02. Data-based conclusion

- **Q-Learning:** fastest initial improvement but plateaus
- **SARSA:** slightly higher final performance than QL
- **DQN:** highest final return (0.7913) but slowest start

03. Key insight

- Performance reflects a learning trade-off
- fast convergence vs stable policy learning vs scalable approximation

Analysis

Algorithm performance depends on: update rule (bias vs variance) , function approximation (generalization vs data efficiency) and training setup (hyperparameters, exploration)

Policy Behavior Differences

- **Q-Learning:** Learns greedy, high-reward-seeking policy early, overcommit to suboptimal actions due to overestimation
- **SARSA:** Learns risk-aware policy aligned with exploration. avoids unstable actions → smoother convergence

Evaluation Rigor

- Multi-seed averaging + standard error
- Compared against baseline (0.78) and upper bound (0.88)
- Learning curves analyzed (not just final values)

Why Learning Curves Differ

- **Q-Learning:** fast but noisy updates
- **SARSA:** lower variance and better stability
- **DQN:** Slower start but stronger asymptotic performance

Gap vs Paper

- Neural network approximation ≠ tabular DQN
- Fewer seeds (3 vs 8) → higher variance

Limitations

Smaller-scale reproduction

- Limited hyperparameter grid
- 3 seeds for tuning, 500k episodes per run
- Returns sensitive to grid & seed choices
- Demonstrates Implementation feasibility

Sparse terminal reward

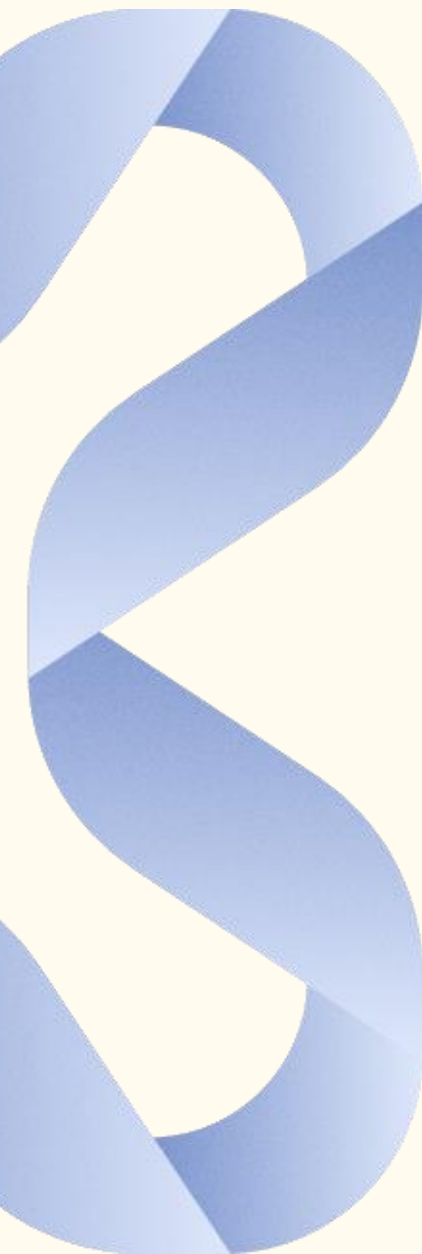
- +1/0 only at episode end
- Slow credit assignment over long trajectories
- Caps learning at our episode budget

Tabular abstraction

- 716 clusters compress 47 continuous features
- Distinct patients can collapse into one state
- No clinical validation of learned policy

Q-Learning $\approx$ SARSA (0.7908 vs 0.7923) — paper's separation not yet emerged at our scale.

Recommendations

- 
- 
- 01.** **Train longer, tune finer**
 - 2–5M episodes per seed
 - Wider α range, randomized search
 - 02.** **Densify the reward**
 - Intermediate clinical signals (SOFA, MAP, lactate)
 - Faster credit assignment
 - 03.** **Safety-aware evaluation**
 - Off-policy evaluation (IS, doubly robust)
 - Constrained RL for clinical plausibility

Summary

- **Both algorithms learn at reproduction scale** 0.79 final return, above random (0.78), below optimal (0.88)
- **Off-policy vs on-policy: little separation** Theoretical gap needs longer training to emerge
- **DQN added complexity, not gain** Function approximation underused in 716-state MDP

Careful tuning of tabular methods > algorithmic sophistication in small clinical MDPs.

The background is a solid light yellow. In the four corners, there are abstract, 3D-style geometric shapes in two shades of blue. The top-left and bottom-right corners feature L-shaped blocks. The top-right and bottom-left corners feature flowing, ribbon-like loops.

**Thank
You**